Taller B: El registro

Qué hace que una cadena de bloques no se pueda reescribir

Integrantes: _____ _____

_____ _____

Profesor: Andrey Lizarazo

Fecha: _____

Resumen del caso

Un usuario acusa al mercado de licencias de haberle borrado una compra; el mercado responde que eso es imposible porque "la cadena verifica". En este informe construimos el mismo registro de dos maneras —con CBC y con Davies–Meyer— y mostramos que:

- la cadena CBC es una biyección conocida la clave: quien la tiene reescribe cualquier historia en microsegundos y el verificador la acepta; peor aún, borrar la primera o la última transferencia ni siquiera requiere la clave;
- la cadena Davies–Meyer no se deja retroceder: el ataque se detiene en una ecuación $E_m(x) \oplus x = h$ que no admite despeje, y que solo sirve si el sello de referencia está publicado fuera del control del mercado.

Conclusión del caso: *el argumento del mercado es falso.* Que la cadena CBC verifique no prueba nada sobre si el mercado la alteró. El usuario tiene razón en que el borrado era posible y además indetectable para el verificador.

Declaración general de variables

Estas declaraciones valen para todo el informe; antes de cada ejercicio se repiten las que intervienen en él.

- $n = 128$: número de bits de un bloque. $\{0,1\}^{128}$: conjunto de bloques de 128 bits (16 bytes).
- $\mathcal{K} = \{0,1\}^{128}$: espacio de claves de AES-128.
- $E : \mathcal{K} \times \{0,1\}^{128} \to \{0,1\}^{128}$, $(k, x) \mapsto E_k(x)$: cifrado AES. Para cada k fija, E_k es una permutación de $\{0,1\}^{128}$ y $D_k = E_k^{-1}$, es decir $D_k(E_k(x)) = x$ y $E_k(D_k(y)) = y$ para todo $x, y \in \{0,1\}^{128}$.
- $\oplus : \{0,1\}^{128} \times \{0,1\}^{128} \to \{0,1\}^{128}$: suma XOR bit a bit. Propiedades que se citarán por nombre:

  (X1) *autoinversa*: $x \oplus x = 0$;
  (X2) *neutro*: $x \oplus 0 = x$;
  (X3) *asociativa y conmutativa.*

- $t \in \mathbb{Z}$, $t \geq 1$: número de entradas del registro (aquí $t = 4$).

- $i \in \{1, \ldots, t\}$: índice de bloque. $m_i \in \{0,1\}^{128}$: entrada i en claro. $c_i \in \{0,1\}^{128}$: bloque cifrado i. $h_i \in \{0,1\}^{128}$: valor acumulado de la cadena tras el bloque i.
- $\|$: concatenación de bloques.

Todo el código está en la carpeta `codigo/`: `registro_tool.py` (las tres funciones pedidas) y `experimentos.py` (las mediciones y ataques). Las salidas reproducidas en este informe son literales.

1. Ejercicio 1. La recurrencia y el error (25 puntos)

Declaración de variables.

- $k \in \mathcal{K}$: clave del mercado, `4ac43a52cc764438cae7b3af9fced5a9`.
- $IV \in \{0,1\}^{128}$: valor de arranque, `3c1cc2d73a499ca72f36e7303d76a130`. Convenio $c_0 := IV$.
- $m_1, \ldots, m_4 \in \{0,1\}^{128}$ y $c_1, \ldots, c_4 \in \{0,1\}^{128}$, ligados por $c_i = E_k(m_i \oplus c_{i-1})$.
- $e \in \{0,1\}^{128}$, $e \neq 0$: máscara de error introducida en un bloque cifrado.
- $c'_3 = c_3 \oplus e \in \{0,1\}^{128}$: bloque c_3 alterado; $m'_3, m'_4 \in \{0,1\}^{128}$: entradas descifradas con el bloque alterado.

(a) Leer el registro

Despeje de la dirección inversa. Partimos de la ecuación del enunciado, $c_i = E_k(m_i \oplus c_{i-1})$. Aplicamos D_k a ambos lados y usamos $D_k \circ E_k = \mathrm{id}$:

$$D_k(c_i) = m_i \oplus c_{i-1}.$$

Sumamos c_{i-1} a ambos lados; por (X3) y (X1), $c_{i-1} \oplus c_{i-1} = 0$, y por (X2):

$$D_k(c_i) \oplus c_{i-1} = m_i \oplus (c_{i-1} \oplus c_{i-1}) = m_i \oplus 0 = m_i.$$

Así, la recurrencia en sentido inverso es $m_i = D_k(c_i) \oplus c_{i-1}$, con $c_0 = IV$.

Implementación. Las dos direcciones, tal como quedaron en `registro_tool.py`:

```
def cbc_descifrar(k, iv, cadena):
    entradas, anterior = [], iv
    for c in bloques(cadena):
        entradas.append(xor(aes_d(k, c), anterior))   # m_i = D_k(c_i) XOR c_{i-1}
        anterior = c
    return entradas

def cbc_cifrar(k, iv, entradas):
    salida, anterior = b"", iv
    for m in entradas:
        assert len(m) == 16
        anterior = aes_e(k, xor(m, anterior))         # c_i = E_k(m_i XOR c_{i-1})
        salida += anterior
    return salida
```

Resultado. `python3 registro_tool.py leer` devuelve:

i	c_i	m_i (hex)	m_i (texto)
1	`acc42eca...a91947`	`545830313a4c303034323a45512d4120`	`TX01:L0042:EQ-A␣`
2	`bcba08d8...b4781f`	`545830323a4c303034323a45512d4220`	`TX02:L0042:EQ-B␣`
3	`7d03a386...74c0cd`	`545830333a4c303137373a45512d4120`	`TX03:L0177:EQ-A␣`
4	`8210d2b6...a52261`	`545830343a4c303137373a45512d4220`	`TX04:L0177:EQ-B␣`

Es decir: la licencia L0042 pasó a EQ-A (TX01) y luego a EQ-B (TX02); la L0177 pasó a EQ-A (TX03) y luego a EQ-B (TX04). Además comprobamos la ida y vuelta:
`cbc_cifrar(k, iv, cbc_descifrar(k, iv, c)) == c` da `True`. El verificador del mercado (`python3 verificador.py cbc registro_cbc.json`) imprime exactamente las mismas cuatro entradas y responde `CADENA VALIDA`.

(b) Romper un byte

1. Experimento. Alteramos el primer byte de c_3 con la máscara $e = \texttt{40}\,\texttt{00}\cdots\texttt{00}$ (un solo bit) y desciframos de nuevo (`python3 registro_tool.py romper 3` y `experimentos.py`). Comparación byte a byte:

Entrada	Original	Tras el daño	Diferencia ($m_i \oplus m_i'$)
1	`545830313a4c...2d4120`	`545830313a4c...2d4120`	0 bytes, 0 bits: **intacta**
2	`545830323a4c...2d4220`	`545830323a4c...2d4220`	0 bytes, 0 bits: **intacta**
3	`545830333a4c...2d4120`	`de2854c98577...57eaad`	16 bytes, 68 bits: **destruida**
4	`545830343a4c...2d4220`	`145830343a4c...2d4220`	1 byte, 1 bit (`0x40`): **alterada**

La diferencia completa de la entrada 3 es `8a7064fabf3ba44a683e6bf6827aab8d`: no guarda ninguna relación visible con e. La de la entrada 4 es `40000000000000000000000000000000`, exactamente e: la `T` (`0x54`) pasó a `0x14` y nada más cambió. Con este registro dañado el verificador responde `CADENA INVALIDA` y marca las entradas 3 y 4.

Por qué (derivación). Con $c_3' = c_3 \oplus e$ y los demás bloques sin tocar:

- m_1, m_2: sus fórmulas $m_1 = D_k(c_1) \oplus IV$ y $m_2 = D_k(c_2) \oplus c_1$ no contienen a c_3; quedan intactas.
- $m_3' = D_k(c_3 \oplus e) \oplus c_2$. El error entra **por dentro** de D_k. Como D_k es una permutación pseudoaleatoria, $D_k(c_3 \oplus e)$ no tiene relación útil con $D_k(c_3)$: cada bit de salida cambia con probabilidad $\approx 1/2$ (observamos 68 de 128). La entrada queda destruida e impredecible para quien no tenga k.
- $m_4' = D_k(c_4) \oplus c_3' = D_k(c_4) \oplus c_3 \oplus e = m_4 \oplus e$ por (X3). El error entra **por fuera**, como sumando del XOR final, y pasa intacto: cambian exactamente los bits de e.
- $m_5, \ldots$ (si existieran) solo dependen de $c_4, c_5, \ldots$ y quedarían intactas: el daño se propaga a dos bloques y se detiene.

2. La asimetría y su consecuencia práctica. En m_3 el error está *dentro* de D_k; en m_4 está *fuera*. Para quien daña a propósito la diferencia es total:

- Sobre m_4 tiene **control exacto y sin clave**: si quiere que m_4 pase a valer m_4^*, le basta usar $e = m_4 \oplus m_4^*$ (es la ecuación $a \oplus x = b$ del Taller 0). Lo comprobamos: con $e = (\texttt{B} \oplus \texttt{A})$ en la posición 14 de c_3, la entrada 4 pasa a decir `TX04:L0177:EQ-A`, sin conocer k.

- Sobre m_3 no tiene ningún control: el precio de su manipulación es que la entrada anterior sale como basura. En este registro esa basura es detectable porque no cumple el formato (la probabilidad de que 16 bytes aleatorios cumplan `TX\d\d:L\d\d\d\d:[A-Z][A-Z]-[A-Z]` es $100 \cdot 10^4 \cdot 26^3/2^{128} \approx 5 \cdot 10^{-29}$).

Es decir: CBC es *maleable*. El adversario sin clave puede dirigir bits de un bloque a costa de destruir el anterior. Lo único que lo delata es la redundancia del formato, no el encadenamiento. Y si el bloque que se sacrifica es el IV (que viaja en claro), ni siquiera hay basura: con $IV' = IV \oplus m_1 \oplus m_1^*$ se cambia m_1 a voluntad y todo lo demás queda intacto.

2. Ejercicio 2. Reescribir la historia (30 puntos)

Declaración de variables.

- $k \in \mathcal{K}$, $IV \in \{0,1\}^{128}$ como en el ejercicio 1; $t \in \mathbb{Z}$, $t \geq 1$.
- $M = (\{0,1\}^{128})^t$: conjunto de historias $(m_1, \ldots, m_t)$. $C = (\{0,1\}^{128})^t$: conjunto de cadenas $(c_1, \ldots, c_t)$.
- $F_{k,IV} : M \to C$, la aplicación de cifrado CBC; $G_{k,IV} : C \to M$, la candidata a inversa.
- $m^* = (m_1^*, \ldots, m_t^*) \in M$: historia alternativa ("lo que quiero que diga").

(a) Hacerlo

Historia falsa: la misma, salvo que TX02 dice que L0042 se quedó en EQ-A:

$$m^* = (\texttt{TX01:L0042:EQ-A}_{\sqcup},\ \texttt{TX02:L0042:EQ-A}_{\sqcup},\ \texttt{TX03:L0177:EQ-A}_{\sqcup},\ \texttt{TX04:L0177:EQ-B}_{\sqcup}).$$

Con `python3 registro_tool.py reescribir` se cifra completa con `cbc_cifrar` y se guarda en `falsificado.json`. Cadena resultante:

i	c_i^* (falsificada)	c_i (original)
1	`acc42ecaebdb06d4e3b2852697a91947`	`acc42ecaebdb06d4e3b2852697a91947`
2	`8f3ebfc015f257f20935a4a9bb070b22`	`bcba08d852933bcb8614195bedb4781f`
3	`3314b26facf1eaae43ea1b86beb9ec3b`	`7d03a3865c756f1ed89c5c56fd74c0cd`
4	`0b1fc3366cb2c953363114fc04286dc5`	`8210d2b695c9514852ac55dcefa52261`

c_1 no cambia porque m_1 no cambió; desde c_2 todo cambia, porque cada bloque arrastra al siguiente. Esto *no* es una protección: quien tiene la clave simplemente recalcula todo.

Tiempo. Promedio de 1000 reescrituras completas: **22 microsegundos** por historia (medido con `time.perf_counter` en `experimentos.py`).

Respuesta literal del verificador. `python3 verificador.py cbc falsificado.json`:

```
Registro leido de la cadena CBC:

  entrada 1: 'TX01:L0042:EQ-A '
  entrada 2: 'TX02:L0042:EQ-A '
  entrada 3: 'TX03:L0177:EQ-A '
  entrada 4: 'TX04:L0177:EQ-B '

>>> CADENA VALIDA. Todas las entradas estan bien formadas
```

```
y el encadenamiento es consistente.

Pregunta para el informe: el verificador acepto este
registro. Eso prueba que nadie lo altero?
```

El verificador no se dio cuenta de nada: acepta la historia falsa con el mismo mensaje con el que acepta la verdadera.

(b) Demostración: la reescritura siempre funciona

1.

Tesis. Fijados $k \in \mathcal{K}$ e $IV \in \{0,1\}^{128}$, la aplicación $F_{k,IV} : (\{0,1\}^{128})^t \to (\{0,1\}^{128})^t$, $(m_1, \ldots, m_t) \mapsto (c_1, \ldots, c_t)$ con $c_0 = IV$ y $c_i = E_k(m_i \oplus c_{i-1})$, es biyectiva, y su inversa es $G_{k,IV}(c_1, \ldots, c_t) = (m_1, \ldots, m_t)$ con $m_i = D_k(c_i) \oplus c_{i-1}$, $c_0 = IV$.

Hipótesis. (H1) $k \in \mathcal{K}$ e $IV \in \{0,1\}^{128}$ son fijos y conocidos. (H2) Para k fija, E_k es una permutación de $\{0,1\}^{128}$ con inversa D_k: $D_k(E_k(x)) = x$ y $E_k(D_k(y)) = y$ para todo $x, y \in \{0,1\}^{128}$. (H3) $(\{0,1\}^{128}, \oplus)$ cumple (X1), (X2), (X3). (H4) $t \geq 1$, y $F_{k,IV}$, $G_{k,IV}$ están definidas por las fórmulas de la tesis. Ambas están bien definidas: cada componente se calcula a partir de datos ya disponibles (c_{i-1} es dato de entrada en G, y en F se calcula en el paso anterior).

Planteamiento. Una aplicación entre dos conjuntos es biyectiva si y solo si tiene inversa a ambos lados. Probaremos (i) $G \circ F = \mathrm{id}_M$ y (ii) $F \circ G = \mathrm{id}_C$. Partimos de la ecuación del enunciado, $c_i = E_k(m_i \oplus c_{i-1})$.

Comparación. *(i)* $G \circ F = \mathrm{id}_M$. Sea $m = (m_1, \ldots, m_t) \in M$ y $c = F(m)$. Sea $(\tilde{m}_1, \ldots, \tilde{m}_t) = G(c)$. Para cada i, con $c_0 = IV$ en ambas aplicaciones:

$$\begin{aligned}
\tilde{m}_i &= D_k(c_i) \oplus c_{i-1} && \text{(definición de } G\text{)} \\
&= D_k\big(E_k(m_i \oplus c_{i-1})\big) \oplus c_{i-1} && \text{(definición de } F\text{)} \\
&= (m_i \oplus c_{i-1}) \oplus c_{i-1} && \text{(H2: } D_k \circ E_k = \mathrm{id}\text{)} \\
&= m_i \oplus (c_{i-1} \oplus c_{i-1}) && \text{(X3: asociatividad)} \\
&= m_i \oplus 0 && \text{(X1: } x \oplus x = 0\text{)} \\
&= m_i && \text{(X2: } x \oplus 0 = x\text{).}
\end{aligned}$$

Observamos que G usa los c_{i-1} *de la cadena* y F usa los c_{i-1} que ella misma produjo: son los mismos porque $c = F(m)$. Por tanto $G(F(m)) = m$.

(ii) $F \circ G = \mathrm{id}_C$. Sea $c = (c_1, \ldots, c_t) \in C$, $m = G(c)$ y $(\tilde{c}_1, \ldots, \tilde{c}_t) = F(m)$, con $\tilde{c}_0 = c_0 = IV$. Probamos por inducción sobre i que $\tilde{c}_i = c_i$. Caso base $i = 0$: $\tilde{c}_0 = IV = c_0$. Paso inductivo: supongamos $\tilde{c}_{i-1} = c_{i-1}$. Entonces

$$\begin{aligned}
\tilde{c}_i &= E_k(m_i \oplus \tilde{c}_{i-1}) && \text{(definición de } F\text{)} \\
&= E_k(m_i \oplus c_{i-1}) && \text{(hipótesis inductiva)} \\
&= E_k\big(D_k(c_i) \oplus c_{i-1} \oplus c_{i-1}\big) && \text{(definición de } G\text{)} \\
&= E_k(D_k(c_i) \oplus 0) = E_k(D_k(c_i)) && \text{(X3, X1, X2)} \\
&= c_i && \text{(H2: } E_k \circ D_k = \mathrm{id}\text{).}
\end{aligned}$$

Luego $F(G(c)) = c$.

Cierre lógico. F tiene inversa por la izquierda (i), luego es inyectiva; tiene inversa por la derecha (ii), luego es sobreyectiva. Una aplicación inyectiva y sobreyectiva es biyectiva, y su inversa es única e igual a G. La demostración no usa ninguna propiedad de los m_i: vale para *toda* historia de longitud t.

Conclusión. Conocidos k e IV, $F_{k,IV}$ es una biyección de $(\{0,1\}^{128})^t$ en sí mismo con inversa explícita $G_{k,IV}$, $m_i = D_k(c_i) \oplus c_{i-1}$. ■

2. Consecuencia. Sí: por la sobreyectividad (en realidad basta con que F esté definida en todo M), para toda historia alternativa $m^* \in M$ existe una cadena que la verifica, a saber $c^* = F_{k,IV}(m^*)$, y por la inyectividad es la única con ese IV. Para construirla hace falta exactamente lo mismo que para cifrar: la clave k y el IV (que es público). No hace falta ninguna búsqueda; es un cálculo directo de t cifrados AES, que medimos en $22\,\mu$s. Como el "verificador" solo comprueba que $G(c^*)$ tenga un formato válido, y $G(c^*) = m^*$ tiene formato válido por construcción, acepta c^* siempre.

(c) Dictamen

Contra qué sí protege. Contra la *corrupción accidental* de los bloques cifrados: un error aleatorio en c_j destruye m_j (ejercicio 1) y, con probabilidad $1 - 5 \cdot 10^{-29}$, la basura no cumple el formato y el verificador lo detecta. También impide que un tercero *sin* la clave lea el registro: es confidencialidad, no integridad.

Contra qué no protege nunca.

- Contra la *alteración deliberada* por quien tiene la clave: es decir, el propio mercado, que es quien custodia el registro. Por (b), cualquier historia se reescribe en microsegundos y verifica.
- Contra un tercero sin clave que *borra extremos*. Lo comprobamos (`sin_ultima.json`, `sin_primera.json`): quitar c_4 deja una cadena de tres entradas válidas; quitar c_1 y poner $IV' = c_1$ deja TX02, TX03, TX04 válidas. En ambos casos el verificador responde `CADENA VALIDA` sin que se haya usado la clave. Es precisamente "borrar una compra".
- Contra la manipulación dirigida de bits sin clave (maleabilidad, ejercicio 1(b)), salvo por la redundancia del formato.

Quién tenía razón. El mercado no. Su argumento —"la cadena verifica, luego no se pudo alterar"— es falso: hemos construido una cadena alterada que verifica, y hemos demostrado que eso es posible para cualquier historia. Que la cadena verifique es compatible tanto con que el mercado sea honesto como con que haya borrado la compra. Con este registro el usuario no puede *probar* que se la borraron, pero el mercado tampoco puede probar que no lo hizo; y la afirmación de que "es imposible" es falsa.

3. Ejercicio 3. La cadena que no se deja (45 puntos)

Declaración de variables.

- $h_0 = 0^{128} \in \{0,1\}^{128}$: valor inicial público. $h_1, \ldots, h_4 \in \{0,1\}^{128}$: valores acumulados.
- $m_1, \ldots, m_4 \in \{0,1\}^{128}$: entradas en claro, *públicas*, que en Davies–Meyer hacen de clave: $m_i \in \mathcal{K}$.
- $f : \mathcal{K} \times \{0,1\}^{128} \to \{0,1\}^{128}$, $f(m,h) = E_m(h) \oplus h$: función de compresión de Davies–Meyer. Recurrencia: $h_i = f(m_i, h_{i-1}) = E_{m_i}(h_{i-1}) \oplus h_{i-1}$.
- $g : \mathcal{K} \times \{0,1\}^{128} \to \{0,1\}^{128}$, $g(m,h) = E_m(h)$: la misma construcción sin el término final.
- $S \in \{0,1\}^{128}$: sello publicado, `51cfb2a7bf6b973b899c232b4ecb521c`.
- $m_2^* =$ `TX02:L0042:EQ-A`$_\sqcup$; h_i^*: valores acumulados con la historia alterada.
- $x \in \{0,1\}^{128}$: incógnita.

(a) Construir y explorar

1. Implementación.

```python
def davies_meyer(h0, entradas):
    h = h0
    for m in entradas:
        assert len(m) == 16
        h = xor(aes_e(m, h), h)                    # el mensaje va como clave
    return h
```

El orden es `aes_e(m, h)`: el bloque de mensaje es la clave de AES y el acumulado es el dato. Valores intermedios, idénticos a los que imprime `python3 verificador.py dm registro_dm.json`:

i	$h_i = E_{m_i}(h_{i-1}) \oplus h_{i-1}$
0	`00000000000000000000000000000000`
1	`692d1a9c017dc9fea6251bf243385b4b`
2	`e0e9a82bb76575a94a4f0153be97c127`
3	`460eaf9791f61ab80f21f1601665fcb7`
4	`51cfb2a7bf6b973b899c232b4ecb521c`

h_4 coincide con el sello de `sello_publicado.txt`: `python3 registro_tool.py sello` responde `COINCIDE`. El sello vive fuera de `registro_dm.json` porque es la referencia contra la que se compara el registro: si estuviera en el mismo archivo, quien edita el archivo editaría también la referencia (ver (c)).

2. El mismo ataque del ejercicio 2. Cambiamos m_2 por m_2^* y recomputamos (`dm_falsificado.json`):

i	h_i^* (historia alterada)	¿igual a h_i?
0	`00000000000000000000000000000000`	sí
1	`692d1a9c017dc9fea6251bf243385b4b`	sí
2	`a1815cf55fe8adaf93fbe63829aef30b`	no
3	`d23ebb39f7eb3a5b140177367483078c`	no
4	`587819e303f7bb5c105313742441de07`	no

El verificador responde `SELLO INCORRECTO. Este registro NO es el publicado.` En CBC el paso siguiente era "recalcular todo y ya". Aquí recalcular es igual de fácil —no hay clave— pero no sirve, porque el resultado tiene que ser igual a S, y S no lo controlamos. Las vías para cerrar el sello y el punto exacto en que cada una se detiene:

1. *Cambiar S.* Imposible: está publicado y replicado fuera del registro.
2. *Volver a la cadena original desde h_2^*.* Habría que encontrar m_2^* con formato válido y $f(m_2^*, h_1) = h_2$, es decir una segunda preimagen de f; pero m_2^* ya está fijado por lo que queremos que diga, y $f(m_2^*, h_1) = h_2^* \neq h_2$.
3. *Corregir con el último bloque.* Conocido h_3^*, buscar m_4' con $E_{m_4'}(h_3^*) \oplus h_3^* = S$, o sea

$$E_{m_4'}(\texttt{d23ebb39...83078c}) = S \oplus h_3^* = \texttt{83f1099e4880ad609d9d541d3a485590}.$$

Aquí nos detenemos. Es encontrar la clave AES que lleva un bloque conocido a otro bloque conocido: no hay despeje, solo búsqueda. Probamos 200 000 claves de formato `TX04:L0177:...`

en 0,99 s (unas 2×10^5 claves/s): ninguna. A ese ritmo, 2^{128} claves serían $\approx 5\times10^{25}$ años. Peor: si m_4' debe cumplir el formato, hay solo $\approx 1{,}8\times10^{10}$ candidatos, y el número esperado de soluciones es $1{,}8\times10^{10}/2^{128}\approx 5\times10^{-29}$: casi seguro *no existe* ninguna.

4. *Retroceder desde el sello.* Para ir hacia atrás desde S con m_4 habría que hallar $x = h_3$ tal que $E_{m_4}(x)\oplus x = S$. Es la ecuación del literal (b)3 y tampoco admite despeje.

Qué información faltaría para seguir. Una forma de invertir f en alguna de sus dos entradas: o bien una inversa de $x \mapsto E_m(x)\oplus x$ (para retroceder desde S), o bien una forma de recuperar la clave de AES a partir de un par (texto, cifrado) (para fabricar el último bloque). Ninguna de las dos existe; y nótese que no hay ningún secreto que robar que lo resuelva: no hay clave.

(b) De qué depende

1. Sin el término final.

Tesis. Sea $h_i = g(m_i, h_{i-1}) = E_{m_i}(h_{i-1})$. Conocidos $h_i \in \{0,1\}^{128}$ y $m_i \in \mathcal{K}$, existe un único $h_{i-1} \in \{0,1\}^{128}$ que satisface la ecuación, y es calculable: $h_{i-1} = D_{m_i}(h_i)$.

Hipótesis. (H1) $m_i \in \mathcal{K}$ y $h_i \in \{0,1\}^{128}$ son conocidos. (H2) Para toda clave $m \in \mathcal{K}$, $E_m : \{0,1\}^{128} \to \{0,1\}^{128}$ es biyectiva, con inversa D_m computable (AES se descifra con la misma eficiencia con que se cifra).

Planteamiento. Partimos de la ecuación $h_i = E_{m_i}(x)$ con incógnita $x \in \{0,1\}^{128}$ y hay que probar dos cosas: existencia de solución y unicidad, más que la solución se puede calcular.

Comparación. *Existencia.* Sea $x_0 := D_{m_i}(h_i)$, que está definido porque D_{m_i} es una función en todo $\{0,1\}^{128}$. Entonces $E_{m_i}(x_0) = E_{m_i}(D_{m_i}(h_i)) = h_i$ por (H2), luego x_0 es solución.
Unicidad. Sean $x, x' \in \{0,1\}^{128}$ soluciones: $E_{m_i}(x) = h_i = E_{m_i}(x')$. Aplicando D_{m_i} a ambos lados y usando $D_{m_i}\circ E_{m_i} = \mathrm{id}$: $x = x'$. (Es la inyectividad de E_{m_i}.)
Despeje. De $h_i = E_{m_i}(h_{i-1})$, aplicando D_{m_i}: $D_{m_i}(h_i) = D_{m_i}(E_{m_i}(h_{i-1})) = h_{i-1}$.
Calculabilidad. $D_{m_i}(h_i)$ es un descifrado AES: una sola operación de coste constante.

Cierre lógico. La solución existe (sobreyectividad), es única (inyectividad) y su fórmula es un descifrado AES, que se ejecuta con los datos de (H1). Ninguno de los pasos requiere información adicional a m_i y h_i.

Conclusión. Sin el término final, cada paso de la cadena se deshace con $h_{i-1} = D_{m_i}(h_i)$. ■

Lo comprobamos en `experimentos.py`: para la variante sin término final, partiendo del sello `44ee89ed...eada27` y descifrando con m_4, m_3, m_2, m_1 se recuperan h_3, h_2, h_1 y llegamos exactamente a $h_0 = 0$.

2. ¿Quién conoce m_i?. Todo el mundo. En una cadena de bloques las entradas son públicas —en `registro_dm.json` están en claro— y *tienen* que serlo: sin ellas nadie podría verificar recomputando. Así, en $h_i = E_{m_i}(h_{i-1})$ la "clave" de la permutación es un dato público; la permutación entera, con su inversa, está en manos de cualquiera. En CBC la reversibilidad estaba restringida a quien tuviera k; aquí se le regala a todos.

Conclusión: **cualquiera** podría retroceder esa cadena, y **no necesita ningún secreto**. Lo demostramos con una falsificación concreta: tomando la historia falsa (TX02 en EQ-A) y retrocediendo desde el sello con $D_{m_4}, D_{m_3}, D_{m_2^*}, D_{m_1}$ obtuvimos $h_0' =$ `4d311f3f3b922efe3b58a6d210b2bb83`, y la historia falsa encadenada desde h_0' da exactamente el mismo sello. Si el punto de partida no está fijado, la falsificación es instantánea. Si h_0 también está fijado, retroceder no basta por sí solo (hay que empalmar la parte que avanza desde h_0 con la que retrocede desde S), pero el empalme es un encuentro a mitad de camino: variando 2^{64} bloques en cada mitad se espera una coincidencia, 2^{64}

operaciones en vez de 2^{128}, y sigue sin hacer falta ningún secreto. Es decir, sin el término final la seguridad de la cadena se derrumba.

3. Con el término final. Para retroceder un paso habría que resolver, para la incógnita $x = h_{i-1}$,

$$E_{m_i}(x) \oplus x = h_i.$$

Intentamos el mismo despeje. Sumar x en ambos lados da $E_{m_i}(x) = h_i \oplus x$, y aplicar D_{m_i} da

$$x = D_{m_i}(h_i \oplus x),$$

con x todavía a los dos lados. Cada movimiento algebraico deja la incógnita en dos sitios: una vez *dentro* de la permutación no lineal E_{m_i} y otra vez *fuera*, sumada. No hay una operación que saque a x de ambos sitios a la vez. La función $x \mapsto E_m(x) \oplus x$ no es una permutación (es la suma de una permutación pseudoaleatoria y la identidad, y se comporta como una función aleatoria: tiene colisiones y valores sin preimagen), así que ni siquiera hay garantía de que la solución exista o sea única, y no se conoce un método mejor que probar valores de x: del orden de 2^{128} intentos.

Comparación con el Taller 0. En $a \oplus x = b$ la incógnita aparece **una sola vez**, y la operación que la envuelve, sumar a, es una traslación del grupo $(\{0,1\}^{128}, \oplus)$ que se deshace sumando a otra vez: $x = a \oplus b$ por (X1) y (X2). La ecuación de Davies–Meyer tiene lo que aquella no tenía: **la incógnita aparece dos veces, una dentro de una permutación no lineal y otra fuera de ella**. El término $\oplus h_{i-1}$, que parecía sobrar, es exactamente lo que vuelve el paso no invertible aunque E_{m_i} y D_{m_i} sean públicas.

(c) Cerrar el caso

	CBC $c_i = E_k(m_i \oplus c_{i-1})$	**Davies–Meyer** $h_i = E_{m_i}(h_{i-1}) \oplus h_{i-1}$
¿Recurrencia reversible?	Sí, para quien tiene k: $m_i = D_k(c_i) \oplus c_{i-1}$ (biyección, ej. 2(b)).	No, para nadie: $E_{m_i}(x) \oplus x = h_i$ no tiene despeje (ej. 3(b)3). Sin el término final sí lo sería, y para todos (ej. 3(b)1–2).
¿Qué hace falta para reescribir la historia?	La clave k: 4 cifrados AES, 22 μs. Para borrar la primera o la última entrada, ni eso.	Invertir la compresión o hallar una clave AES para un par dado: $\approx 2^{128}$ trabajo, y con formato válido casi seguro no hay solución. O bien sustituir el sello, que debe estar fuera de su alcance.
¿Qué protege?	Confidencialidad de las entradas frente a quien no tiene k; detecta corrupción accidental (vía formato). No da integridad.	Integridad de la historia completa frente al sello publicado. No da confidencialidad: las entradas están en claro.
¿Contra qué adversario?	Contra un tercero sin clave que quiera *leer*. No contra el custodio (tiene k), ni contra un tercero que borre extremos o voltee bits.	Contra cualquiera, *incluido el custodio*, siempre que el sello esté publicado fuera de su control y la historia se compare contra él.

¿Y si el sello estuviera dentro de `registro_dm.json`?. Entonces el mercado sí saldría impune. Procedimiento exacto:

1. Editar la entrada: $m_2 \leftarrow m_2^* =$ `TX02:L0042:EQ-A`$_{\sqcup}$.
2. Recomputar hacia delante desde h_0 con la recurrencia, que no requiere clave: $h_1^*, h_2^*, h_3^*, h_4^* =$ `587819e303f7bb5c105313742441de07`.
3. Escribir ese h_4^* como el nuevo sello dentro del archivo.
4. El verificador recomputa, obtiene h_4^*, lo compara con el sello del propio archivo y responde "sello correcto".

Lo hicimos (`dm_con_sello_interno.json`, idéntico al `atk.json` recibido): un verificador que leyera el sello del archivo lo acepta; contra el sello publicado, se rechaza. El cálculo cuesta lo mismo que una verificación honesta. La función de compresión garantiza que *a una historia le corresponde un único sello* en la práctica; no garantiza *cuál es el sello correcto*. Eso lo da el ancla.

Para que un registro sea realmente inmutable hace falta, además de la compresión no invertible:

- un **ancla externa**: el sello publicado y replicado donde el custodio no pueda cambiarlo (otros nodos, una publicación con fecha, un servicio de sellado de tiempo, los propios usuarios guardando copia);
- **publicación oportuna**: el sello de la historia que incluye una transacción debe salir antes de que nadie tenga motivo para alterarla, y quien compra debe poder quedarse con él;
- fijar públicamente el punto de partida h_0 y la longitud de la historia, para que no se pueda empezar en otro sitio ni truncar.

4. La frase que hay que corregir

La frase "una cadena de bloques usa CBC" es engañosa porque confunde *la forma de la recurrencia* con *la propiedad que la hace útil*. Las dos fórmulas encadenan, sí, pero CBC encadena con una permutación reversible para quien tiene la clave, y eso la hace un mecanismo de confidencialidad: el custodio puede descifrar, editar y volver a cifrar sin dejar rastro. Una cadena de bloques necesita lo contrario: que nadie, ni siquiera quien la lleva, pueda retroceder.

Formulación correcta: una cadena de bloques encadena cada bloque con el *hash* del anterior, calculado con una función de compresión resistente a preimágenes y colisiones (como Davies–Meyer sobre un cifrado de bloque), que no se descifra sino que se verifica recomputando; y su inmutabilidad depende además de que el valor final esté anclado en un lugar público que el custodio no controla.

5. Informe final

5.1. Las demostraciones

Están en los ejercicios 2(b)1 (biyectividad de CBC) y 3(b)1 (reversibilidad de la variante sin término final), cada una con sus pasos rotulados: Hipótesis, Planteamiento, Comparación, Cierre lógico y Conclusión, y con la tesis enunciada en recuadro. El análisis de 3(b)3 explica por qué no hay demostración análoga para Davies–Meyer.

5.2. Código

`registro_tool.py` (con las tres funciones completadas)

```python
#!/usr/bin/env python3
"""
================================================================
  HERRAMIENTA DEL REGISTRO
================================================================

La parte aburrida ya esta resuelta: abrir los archivos, leer los
campos, partir en bloques y guardar. Ustedes escriben las funciones
de criptografia, que son el ejercicio.

COMO SE USA
-----------
  1. Pongan este archivo junto a registro_cbc.json y registro_dm.json
  2. Miren que hay adentro:   python3 registro_tool.py ver
  3. Completen las funciones marcadas mas abajo, en este orden:
        cbc_descifrar   -> para leer el registro       (ejercicio 1)
        cbc_cifrar      -> para reescribir la historia (ejercicio 2)
        davies_meyer    -> para la otra cadena         (ejercicio 3)
  4. Vayan probando:
        python3 registro_tool.py leer
        python3 registro_tool.py romper 3
        python3 registro_tool.py reescribir
        python3 registro_tool.py sello
  5. Comprueben con el verificador del mercado:
        python3 verificador.py cbc falsificado.json

Nunca se modifican los archivos originales.
"""

import sys, json
from cripto import aes_e, aes_d, xor, bloques

CBC = "registro_cbc.json"
DM  = "registro_dm.json"

HISTORIA_FALSA = [b"TX01:L0042:EQ-A ", b"TX02:L0042:EQ-A ",
                  b"TX03:L0177:EQ-A ", b"TX04:L0177:EQ-B "]


def cargar_cbc():
    d = json.load(open(CBC))
    return bytes.fromhex(d["clave"]), bytes.fromhex(d["iv"]), bytes.fromhex(d["cadena"])


def cargar_dm():
    d = json.load(open(DM))
    sello = None
    for linea in open("sello_publicado.txt"):
        linea = linea.strip()
        if linea and not linea.startswith("#"):
            sello = linea; break
    return bytes.fromhex(d["h0"]), [e.encode("latin-1") for e in d["entradas"]], sello


# ================================================================
#   LO QUE TIENEN QUE ESCRIBIR
# ================================================================

def cbc_descifrar(k, iv, cadena):
```

```
    """Recupera la lista de entradas a partir de la cadena cifrada.

    La recurrencia es   c_i = E_k( m_i XOR c_{i-1} )   con c_0 = IV.
    Ustedes ya despejaron m_i en el Taller 0.

    Herramientas: aes_d(k, bloque), xor(a, b), bloques(datos)
    Devuelve: una lista de bloques de 16 bytes.
    """
    entradas, anterior = [], iv
    for c in bloques(cadena):
        entradas.append(xor(aes_d(k, c), anterior))   # m_i = D_k(c_i) XOR c_{i-1}
        anterior = c
    return entradas


def cbc_cifrar(k, iv, entradas):
    """Construye la cadena a partir de una lista de entradas.

    Es la recurrencia tal como esta escrita, sin despejar nada.
    Herramientas: aes_e(k, bloque), xor(a, b)
    Devuelve: los bloques cifrados pegados, en bytes.
    """
    salida, anterior = b"", iv
    for m in entradas:
        assert len(m) == 16
        anterior = aes_e(k, xor(m, anterior))         # c_i = E_k(m_i XOR c_{i-1})
        salida += anterior
    return salida


def davies_meyer(h0, entradas):
    """Encadena las entradas con  h_i = E_{m_i}( h_{i-1} ) XOR h_{i-1}

    OJO con el orden de los argumentos: el MENSAJE va como clave.
    Es aes_e(m, h), no aes_e(h, m). Es el error mas comun.

    Devuelve: el valor final, 16 bytes.
    """
    h = h0
    for m in entradas:
        assert len(m) == 16
        h = xor(aes_e(m, h), h)                       # el mensaje va como clave
    return h


# ============================================================

def _falta(e):
    print("\n  %s\n" % e); return None


def cmd_ver():
    k, iv, c = cargar_cbc()
    print("\n  === registro_cbc.json ===")
    print("    clave  : %s   <- si, se las damos a proposito" % k.hex())
    print("    iv     : %s" % iv.hex())
    print("    cadena : %d bloques de 16 bytes" % (len(c) // 16))
    for i, b in enumerate(bloques(c), 1):
        print("       c%d = %s" % (i, b.hex()))
    h0, ents, sello = cargar_dm()
```

```
    print("\n  === registro_dm.json ===")
    print("    h0          : %s" % h0.hex())
    print("    (el sello NO esta en este archivo: esta en sello_publicado.txt)")
    print("    sello publicado : %s" % sello)
    print("    entradas    : (estan EN CLARO, aqui no se oculta nada)")
    for i, m in enumerate(ents, 1):
        print("       m%d = %r" % (i, m.decode("latin-1")))
    print()


def cmd_leer():
    k, iv, c = cargar_cbc()
    try:
        ents = cbc_descifrar(k, iv, c)
    except NotImplementedError as e:
        return _falta(e)
    print("\n  Registro recuperado:")
    for i, m in enumerate(ents, 1):
        try:
            print("    entrada %d: %r" % (i, m.decode("latin-1")))
        except Exception:
            print("    entrada %d: %s  (ilegible)" % (i, m.hex()))
    print()


def cmd_romper(j):
    k, iv, c = cargar_cbc()
    try:
        bien = cbc_descifrar(k, iv, c)
    except NotImplementedError as e:
        return _falta(e)
    bl = bloques(c)
    bl[j - 1] = bytes([bl[j - 1][0] ^ 0x40]) + bl[j - 1][1:]
    mal = cbc_descifrar(k, iv, b"".join(bl))
    print("\n  Dañando el primer byte de c%d:\n" % j)
    for i, (a, b) in enumerate(zip(bien, mal), 1):
        dif = sum(1 for x, y in zip(a, b) if x != y)
        est = "intacta" if dif == 0 else ("alterada en %d byte" % dif if dif == 1 else "destruida (%d
     bytes cambiados)" % dif)
        print("    entrada %d: %-28s %s" % (i, est, b.hex()[:24] + "..."))
    print()


def cmd_reescribir():
    k, iv, c = cargar_cbc()
    try:
        nueva = cbc_cifrar(k, iv, HISTORIA_FALSA)
    except NotImplementedError as e:
        return _falta(e)
    d = json.load(open(CBC)); d["cadena"] = nueva.hex()
    json.dump(d, open("falsificado.json", "w"), indent=2)
    print("\n  Historia escrita:")
    for m in HISTORIA_FALSA:
        print("    %r" % m.decode("latin-1"))
    print("\n  Escrito: falsificado.json")
    print("  Ahora pasenlo por:  python3 verificador.py cbc falsificado.json\n")


def cmd_sello():
    h0, ents, esperado = cargar_dm()
```

```
    try:
        h = davies_meyer(h0, ents)
    except NotImplementedError as e:
        return _falta(e)
    print("\n  sello calculado por ustedes : %s" % h.hex())
    print("  sello publicado por el mercado: %s" % esperado)
    print("\n  %s\n" % ("COINCIDE. Su implementacion esta bien."
                        if h.hex() == esperado else
                        "NO COINCIDE. Revisen el orden de los argumentos de aes_e."))


if __name__ == "__main__":
    a = sys.argv[1:] or ["ayuda"]
    if   a[0] == "ver":        cmd_ver()
    elif a[0] == "leer":       cmd_leer()
    elif a[0] == "romper":     cmd_romper(int(a[1]) if len(a) > 1 else 3)
    elif a[0] == "reescribir": cmd_reescribir()
    elif a[0] == "sello":      cmd_sello()
    else:                      print(__doc__)
```

experimentos.py

```
#!/usr/bin/env python3
"""Experimentos del Taller B que no cubre registro_tool.py.

    python3 experimentos.py

Genera:
  falsificado.json          cadena CBC reescrita (ejercicio 2)
  dm_falsificado.json       registro DM con TX02 alterado (ejercicio 3a)
  dm_con_sello_interno.json registro DM falso con su propio sello (ejercicio 3c)
"""

import json, time
from cripto import aes_e, aes_d, xor, bloques
from registro_tool import (cargar_cbc, cargar_dm, cbc_cifrar, cbc_descifrar,
                           davies_meyer, HISTORIA_FALSA)


def cadena_dm(h0, entradas):
    """Todos los valores intermedios h_0, ..., h_t."""
    hs = [h0]
    for m in entradas:
        hs.append(xor(aes_e(m, hs[-1]), hs[-1]))
    return hs


def titulo(t):
    print("\n" + "=" * 64 + "\n  " + t + "\n" + "=" * 64)


# ---------------------------------------------------------------
titulo("Ejercicio 1(a): ida y vuelta CBC")
k, iv, c = cargar_cbc()
m = cbc_descifrar(k, iv, c)
for i, x in enumerate(m, 1):
    print("  m%d = %r  (hex %s)" % (i, x.decode("latin-1"), x.hex()))
print("  cbc_cifrar(cbc_descifrar(c)) == c :", cbc_cifrar(k, iv, m) == c)
```

```
37
38  # ------------------------------------------------------------------
39  titulo("Ejercicio 1(b): un byte de c3 alterado (XOR 0x40 en el byte 0)")
40  bl = bloques(c)
41  bl[2] = bytes([bl[2][0] ^ 0x40]) + bl[2][1:]
42  mal = cbc_descifrar(k, iv, b"".join(bl))
43  for i, (a, b) in enumerate(zip(m, mal), 1):
44      d = xor(a, b)
45      print("  entrada %d: original %s" % (i, a.hex()))
46      print("             danada   %s" % b.hex())
47      print("             XOR      %s  (%d bytes y %d bits distintos)"
48            % (d.hex(), sum(x != 0 for x in d), sum(bin(x).count("1") for x in d)))
49
50  # ------------------------------------------------------------------
51  titulo("Ejercicio 2(a): reescribir la historia CBC")
52  N = 1000
53  t0 = time.perf_counter()
54  for _ in range(N):
55      nueva = cbc_cifrar(k, iv, HISTORIA_FALSA)
56  dt = (time.perf_counter() - t0) / N
57  d = json.load(open("registro_cbc.json")); d["cadena"] = nueva.hex()
58  json.dump(d, open("falsificado.json", "w"), indent=2)
59  print("  cadena falsificada:")
60  for i, b in enumerate(bloques(nueva), 1):
61      print("    c%d* = %s   (original %s)" % (i, b.hex(), bloques(c)[i - 1].hex()))
62  print("  tiempo medio de la reescritura: %.1f microsegundos (promedio de %d)" % (dt * 1e6, N))
63
64  # ------------------------------------------------------------------
65  titulo("Ejercicio 3(a): Davies-Meyer y el intento de ataque")
66  h0, ents, sello = cargar_dm()
67  hs = cadena_dm(h0, ents)
68  for i, h in enumerate(hs):
69      print("  h%d  = %s" % (i, h.hex()))
70  print("  sello publicado = %s   coincide: %s" % (sello, hs[-1].hex() == sello))
71
72  falsas = list(ents); falsas[1] = b"TX02:L0042:EQ-A "
73  hf = cadena_dm(h0, falsas)
74  print("\n  Con TX02 -> EQ-A:")
75  for i, h in enumerate(hf):
76      print("  h%d* = %s   %s" % (i, h.hex(), "igual" if h == hs[i] else "DISTINTO"))
77  print("  sello publicado = %s" % sello)
78  d = json.load(open("registro_dm.json")); d["entradas"][1] = "TX02:L0042:EQ-A "
79  json.dump(d, open("dm_falsificado.json", "w"), indent=2)
80
81  # El atacante necesita, para seguir, un m4' con E_{m4'}(h3*) = sello XOR h3*.
82  # Es decir, una clave AES que lleve un bloque conocido a otro conocido.
83  objetivo = xor(bytes.fromhex(sello), hf[3])
84  print("\n  Para cerrar con m4' haria falta E_{m4'}(%s) = %s" % (hf[3].hex(), objetivo.hex()))
85  intentos, t0 = 200000, time.perf_counter()
86  for j in range(intentos):
87      cand = b"TX04:L0177:" + j.to_bytes(5, "big")
88      if aes_e(cand, hf[3]) == objetivo:
89          print("  encontrado", cand); break
90  dt = time.perf_counter() - t0
91  tasa = intentos / dt
92  print("  %d claves probadas en %.2f s (%.0f claves/s): ninguna sirve" % (intentos, dt, tasa))
93  print("  tiempo esperado para 2^128 claves: %.1e anos" % (2**128 / tasa / 3.156e7))
94
95  # ------------------------------------------------------------------
96  titulo("Ejercicio 3(b): sin el termino final, la cadena SI se deja")
```

```
def sin_ff(h0, entradas):
    h = h0
    for m in entradas:
        h = aes_e(m, h)
    return h
S = sin_ff(h0, ents)
print("  sello de la variante h_i = E_{m_i}(h_{i-1}): %s" % S.hex())
# retroceder: h_{i-1} = D_{m_i}(h_i), con m_i publico
h = S
for i in range(4, 0, -1):
    h = aes_d(ents[i - 1], h)
    print("  retrocediendo con m%d (publico): h%d = %s" % (i, i - 1, h.hex()))
print("  se recupera h0:", h == h0)
# Falsificacion con punto de partida libre: retroceder la historia falsa
h = S
for mm in reversed(falsas):
    h = aes_d(mm, h)
print("  h0' que hace que la historia FALSA de el mismo sello: %s" % h.hex())
print("  comprobacion sin_ff(h0', falsas) == S :", sin_ff(h, falsas) == S)

# ------------------------------------------------------------------
titulo("Ejercicio 3(c): sello guardado dentro del propio archivo")
hfin = davies_meyer(h0, falsas)
d = json.load(open("registro_dm.json"))
d["entradas"][1] = "TX02:L0042:EQ-A "
d["sello_final"] = hfin.hex()
json.dump(d, open("dm_con_sello_interno.json", "w"), indent=2)
print("  historia falsa recalculada, sello nuevo: %s" % hfin.hex())
print("  un verificador que leyera el sello del archivo diria: %s"
      % ("SELLO CORRECTO" if davies_meyer(h0, [e.encode("latin-1") for e in d["entradas"]]).hex()
         == d["sello_final"] else "SELLO INCORRECTO"))
print("  contra el sello publicado: %s" % ("CORRECTO" if hfin.hex() == sello else "INCORRECTO"))

# ------------------------------------------------------------------
titulo("Ejercicio 2(c): borrar una compra SIN la clave")
d = json.load(open("registro_cbc.json"))
d["cadena"] = c[:48].hex()                        # se quita c4
json.dump(d, open("sin_ultima.json", "w"), indent=2)
d = json.load(open("registro_cbc.json"))
d["iv"], d["cadena"] = c[:16].hex(), c[16:].hex()  # se quita c1 y su IV pasa a ser c1
json.dump(d, open("sin_primera.json", "w"), indent=2)
print("  sin_ultima.json  : se borro TX04 sin usar la clave")
print("  sin_primera.json : se borro TX01 sin usar la clave (IV' = c1)")
print("  pasenlos por:  python3 verificador.py cbc sin_ultima.json")
```

5.3. Cadena falsificada y salida del verificador

Archivo `falsificado.json` (ejercicio 2; el campo `cadena`, que en el archivo es una sola línea de 128 caracteres hexadecimales, se parte aquí en sus cuatro bloques):

```
{
  "modo": "CBC",
  "formula": "c_i = E_k(m_i XOR c_{i-1})",
  "clave": "4ac43a52cc764438cae7b3af9fced5a9",
  "iv": "3c1cc2d73a499ca72f36e7303d76a130",
  "cadena": "acc42ecaebdb06d4e3b2852697a91947
             8f3ebfc015f257f20935a4a9bb070b22
             3314b26facf1eaae43ea1b86beb9ec3b
             0b1fc3366cb2c953363114fc04286dc5"
```

```
}
```

Salida literal de `python3 verificador.py cbc falsificado.json`:

```
Registro leido de la cadena CBC:

  entrada 1: 'TX01:L0042:EQ-A '
  entrada 2: 'TX02:L0042:EQ-A '
  entrada 3: 'TX03:L0177:EQ-A '
  entrada 4: 'TX04:L0177:EQ-B '

>>> CADENA VALIDA. Todas las entradas estan bien formadas
    y el encadenamiento es consistente.

    Pregunta para el informe: el verificador acepto este
    registro. Eso prueba que nadie lo altero?
```

Para contraste, la salida de `python3 verificador.py dm dm_falsificado.json` (la misma alteración sobre Davies–Meyer):

```
Recomputando la cadena Davies-Meyer:

  h0 = 00000000000000000000000000000000
  h1 = E_{m1}(h0) XOR h0 = 692d1a9c017dc9fea6251bf243385b4b
  h2 = E_{m2}(h1) XOR h1 = a1815cf55fe8adaf93fbe63829aef30b
  h3 = E_{m3}(h2) XOR h2 = d23ebb39f7eb3a5b140177367483078c
  h4 = E_{m4}(h3) XOR h3 = 587819e303f7bb5c105313742441de07

  sello recomputado de este archivo : 587819e303f7bb5c105313742441de07
  sello PUBLICADO por el mercado    : 51cfb2a7bf6b973b899c232b4ecb521c

>>> SELLO INCORRECTO. Este registro NO es el publicado.

    Dos cosas que conviene notar:

    1. Aqui NO se uso ninguna clave: solo se recomputo.
    2. El sello de referencia NO sale de este archivo, sino de
       sello_publicado.txt. Si saliera del mismo archivo que se
       esta verificando, cualquiera podria cambiar una entrada,
       recalcular y sustituir tambien el sello.
```

5.4. Tabla comparativa

Ver la tabla del ejercicio 3(c).

5.5. Dictamen para el usuario

Estimado usuario:

Usted preguntó si el mercado pudo haber borrado su compra. La respuesta corta es: **sí, pudo**, y con el sistema que usa hoy nadie lo habría notado.

El mercado guarda el registro en una especie de caja fuerte encadenada, en la que cada transferencia queda atada a la anterior. Eso suena muy seguro, pero el candado de esa caja tiene una sola llave, y la llave la tiene el propio mercado. Con ella puede abrir el registro, cambiar o quitar lo que quiera, volver a cerrarlo y dejarlo exactamente igual de "encadenado" que antes. Nosotros lo hicimos: cambiamos una transferencia en una fracción de segundo y la herramienta oficial del mercado dijo que todo estaba en

orden. Incluso sin la llave se puede quitar la primera o la última transferencia sin que la herramienta proteste.

Por eso, cuando el mercado dice "la cadena verifica, así que no pudo pasar", está diciendo algo que no es cierto. Que la herramienta diga que todo está bien no demuestra que nadie haya tocado el registro. Esto no prueba que el mercado le haya borrado su compra; prueba que su defensa no vale.

Para que la respuesta fuera "no, imposible", el mercado tendría que cambiar dos cosas:

1. Dejar de usar un candado con llave y usar un "sello": una huella del registro completo que cualquiera puede recalcular, que cambia por completo si se toca una sola letra, y que nadie —ni siquiera el mercado— sabe fabricar para un registro distinto.
2. Publicar esa huella en un sitio que el mercado no controle (por ejemplo, entregársela a cada comprador en su recibo, o publicarla en varios lugares independientes). Si el mercado guarda la huella en su propio cajón, puede cambiar el registro y la huella a la vez.

Con esas dos condiciones, usted guardaría la huella del día de su compra y, si alguien borrara su transferencia, la huella dejaría de coincidir y cualquiera podría comprobarlo.

5.6. Media página por integrante: por qué "una cadena de bloques usa CBC" es engañoso

Nota: cada integrante debe redactar o revisar su propia sección. Lo que sigue son cuatro enfoques distintos como punto de partida.

Integrante 1: _____ — La forma no es la propiedad

Las fórmulas $c_i = E_k(m_i \oplus c_{i-1})$ y $h_i = H(B_i, h_{i-1})$ se parecen porque las dos son recurrencias: cada término depende del anterior. Pero "depender del anterior" es una propiedad de la forma, y la seguridad no está en la forma sino en la función que se aplica en cada paso. En CBC esa función es una permutación que, con la clave, se deshace; demostramos que la aplicación historia→cadena es una biyección con inversa explícita. Un objeto biyectivo y calculable en ambos sentidos no puede servir de prueba de nada: cada historia tiene su cadena y cada cadena su historia, y pasar de una a otra cuesta microsegundos. Decir que una cadena de bloques "usa CBC" es tomar el parecido de las fórmulas por un parecido de propiedades. Lo correcto: una cadena de bloques encadena con una función de compresión no invertible, y se verifica recomputando, no descifrando.

Integrante 2: _____ — ¿Quién tiene la llave?

En CBC la única barrera para reescribir la historia es la clave, y en un registro la clave la tiene precisamente quien lo custodia: el adversario del que queremos protegernos. CBC protege contra terceros que quieren *leer*, no contra el custodio que quiere *editar*. Además mostramos que, aun sin la clave, borrar la primera o la última entrada deja una cadena que el verificador acepta. Una cadena de bloques invierte el planteamiento: no hay llave, las entradas son públicas y la protección no consiste en ocultar sino en que nadie pueda fabricar una historia distinta con el mismo sello. Por eso la formulación correcta no menciona cifrado: una cadena de bloques es una cadena de hashes, cuya seguridad descansa en la dificultad de invertir la compresión, y no en un secreto.

Integrante 3: _____ — Un solo término

El ejercicio 3 muestra que la diferencia puede ser un solo término. Davies–Meyer sin el $\oplus h_{i-1}$ final es $h_i = E_{m_i}(h_{i-1})$, una permutación de h_{i-1} cuya "clave" es el mensaje, que es público; cualquiera la

retrocede con D_{m_i}. Esa variante sí "se parece a CBC" en lo esencial —es reversible— y es insegura por eso. Con el término final, la ecuación $E_{m_i}(x) \oplus x = h_i$ tiene la incógnita dentro y fuera de una permutación y no se despeja. La moraleja es que lo que hace a una cadena de bloques no es usar un cifrado de bloque (Davies–Meyer usa AES), sino usarlo de forma que el resultado no sea invertible. Decir "usa CBC" esconde justo el detalle que importa.

Integrante 4: _____ — El ancla

Incluso con una función no invertible, el ejercicio 3(c) muestra que si el sello vive dentro del archivo, el custodio cambia una entrada, recalcula y reemplaza el sello, sin ninguna clave. La función de compresión solo garantiza que a cada historia le corresponde un sello; no dice cuál es el correcto. Eso lo decide un valor publicado que el custodio no controla. En una cadena de bloques ese papel lo cumplen la replicación y el consenso entre muchos participantes. Por eso la frase "una cadena de bloques usa CBC" es doblemente engañosa: atribuye la seguridad a un modo de cifrado, cuando proviene de dos cosas que CBC no tiene: una compresión que no se deja invertir y un ancla pública fuera del alcance de quien lleva el registro.